

Building a Secure Chat Application

Phase 0 — Deployment Environment

Phase 1 — Baseline Chat Application

A one-to-one chat application over TCP sockets, to be hardened across five phases. This report covers the deployment environment and the baseline plaintext application together with its verification.

Four virtual machines were provisioned on an isolated virtual network and connectivity verified before any application code was run. The application is written in C++17 over POSIX sockets, with a length-prefixed record protocol chosen so that the framing remains valid once message bodies stop being printable text. Phase 1 deliberately provides no confidentiality: the server logs the full content of every message it relays, and a packet capture taken on the network segment shows the entire conversation in cleartext. That is the baseline the remaining phases must dismantle.

Bathula Rithvik

23B0939

September 4, 2026

Contents

1	Introduction	2
2	Phase 0 — Deployment Environment	2
2.1	Requirement	2
2.2	Topology	2
2.3	Design notes	2
2.4	Connectivity verification	3
2.5	Toolchain	4
3	Phase 1 — Baseline Chat Application	4
3.1	Requirements	4
3.2	Protocol design	4
3.3	Implementation	6
3.4	Verification	7
3.5	Evidence index	9
3.6	Deliverables checklist	9
4	Building and Running	9
5	Next Phase	10
A	Generative AI Usage	10

1 Introduction

This report documents the first stage of a one-to-one chat application built over TCP sockets and progressively hardened across five phases. It covers the deployment environment (Phase 0) and the baseline plaintext chat application together with its verification (Phase 1).

The application is written in C++17 using POSIX sockets only. No TLS/SSL library is used anywhere: `<openssl/ssl.h>` and `<openssl/dh.h>` are excluded by design, as required. Phase 1 introduces no cryptography at all — its purpose is to establish the baseline against which every later phase is measured, and specifically to demonstrate that an unprotected chat protocol is fully readable both by the relaying server and by anyone capturing traffic on the network segment.

One decision shaped the implementation more than any other. The simplest framing that satisfies Phase 1 would have to be discarded as soon as message bodies stop being printable text, so a binary-safe format was adopted from the start instead. Section 3.2 sets out that choice and the reasoning behind it.

2 Phase 0 — Deployment Environment

2.1 Requirement

The assignment requires the application to be implemented and demonstrated across separate virtual machines rather than on a single machine over loopback, so that packet captures and later man-in-the-middle demonstrations are meaningful. The minimum topology is one server VM and two client VMs on a common virtual network; a fourth VM is required for the attack tasks in Phases 2 and 3.

2.2 Topology

Four Ubuntu 26.04.1 LTS Server virtual machines run under VirtualBox 7.2.16 on an Ubuntu 24.04 host. Each VM has two network adapters, serving distinct purposes:

- **Adapter 1 — NAT** (`enp0s3`). Provides outbound internet access for package installation, and inbound SSH from the host through a loopback-bound port forward. It is not part of the assignment network.
- **Adapter 2 — Internal Network** `secure-chat` (`enp0s8`). A private virtual Ethernet switch connecting only these four VMs. All chat traffic, all packet captures, and all later attack traffic occur here.

Role	VM	secure-chat IP	MAC (Adapter 2)	RAM	Purpose
S	vm1-server	10.10.0.10	08:00:27:11:08:0E	2048 MB	chat server
C1	vm2-client1	10.10.0.11	08:00:27:2A:3C:7B	1536 MB	client
C2	vm3-client2	10.10.0.12	08:00:27:32:22:C2	1536 MB	client
M	vm4-mallory	10.10.0.13	08:00:27:26:94:51	1536 MB	MITM proxy (Phases 2–3)

Table 1. Virtual machine roles and addressing on the `secure-chat` network (10.10.0.0/24).

2.3 Design notes

Figure 1 shows how the two adapters divide the work.

Why two adapters. VirtualBox NAT creates a *separate, isolated* network per virtual machine rather than one shared network. Consequently every VM sees itself as 10.0.2.15, with gateway

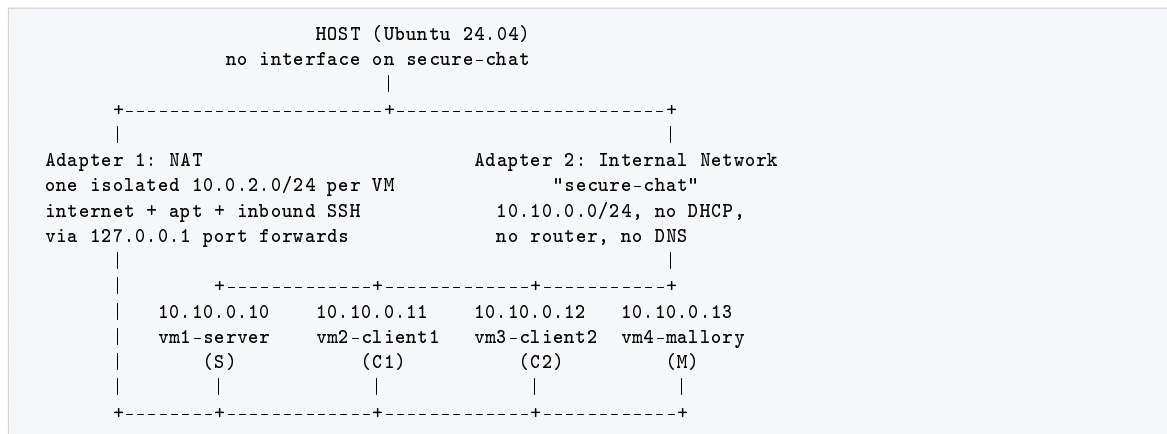


Figure 1. VM topology. The chat service listens on TCP port 5555 on 10.10.0.10.

10.0.2.2 — the identical addressing is correct, not a misconfiguration, because those addresses have meaning only inside each VM’s own NAT network. The direct consequence is that VMs cannot reach one another over NAT at all, which is precisely why the second adapter exists.

Why static addressing. A VirtualBox Internal Network is a bare virtual switch with no DHCP server, so `enp0s8` comes up with only an IPv6 link-local address and no IPv4. Addresses are therefore assigned statically through a dedicated netplan file, `/etc/netplan/99-secure-chat.yaml`, kept separate from the installer’s configuration so that an error can never break the interface carrying the SSH session:

```

network:
  version: 2
  ethernets:
    enp0s8:
      dhcp4: false
      addresses:
        - 10.10.0.10/24      # .11 on C1, .12 on C2, .13 on M

```

No default gateway and no nameservers are configured on this interface. The `secure-chat` network has no router, and a second default route would compete with the NAT adapter’s, potentially handing internet traffic to a network with nothing to forward it.

Capture vantage point. The host has no interface on `secure-chat` and therefore cannot observe chat traffic at all. All packet capture runs inside the VMs with `tshark -i enp0s8`; the resulting `.pcap` files are copied to the host for analysis in the Wireshark GUI.

2.4 Connectivity verification

The assignment requires basic connectivity between VMs to be verified with `ping` before the chat application is run. A full 4×4 matrix was executed, each VM pinging all four addresses:

=== from s ===	=== from c1 ===	=== from c2 ===	=== from m ===
10.10.0.10 OK	10.10.0.10 OK	10.10.0.10 OK	10.10.0.10 OK
10.10.0.11 OK	10.10.0.11 OK	10.10.0.11 OK	10.10.0.11 OK
10.10.0.12 OK	10.10.0.12 OK	10.10.0.12 OK	10.10.0.12 OK
10.10.0.13 OK	10.10.0.13 OK	10.10.0.13 OK	10.10.0.13 OK

All 16 checks succeeded. A representative single probe confirms the peers are directly on-link:

```

$ ping -c1 -W1 10.10.0.11
64 bytes from 10.10.0.11: icmp_seq=1 ttl=64 time=0.295 ms

```

The TTL of 64 is unchanged from the Linux default, confirming zero router hops — as expected on a single switched segment. Layer-2 adjacency was confirmed independently: the MAC addresses resolved by ARP on the server match the VirtualBox adapter configuration exactly, so each address provably belongs to the intended virtual machine.

Full environment records are in `evidence/phase0/`.

2.5 Toolchain

Identical on all four VMs: g++ 15.2.0, OpenSSL 3.5.5 with `libssl-dev` headers (needed from Phase 2 for `bn.h`, `evp.h` and `x509.h`), and TShark 4.6.4. Binaries are compiled on the VMs rather than on the host, since the host runs OpenSSL 3.0.13.

3 Phase 1 — Baseline Chat Application

3.1 Requirements

Phase 1 requires a chat server supporting exactly one server and two clients connected simultaneously over TCP, with messages typed by C1 delivered to C2 through the server and vice versa, relayed as-is with no encryption, and the command interface implemented from this phase onward.

3.2 Protocol design

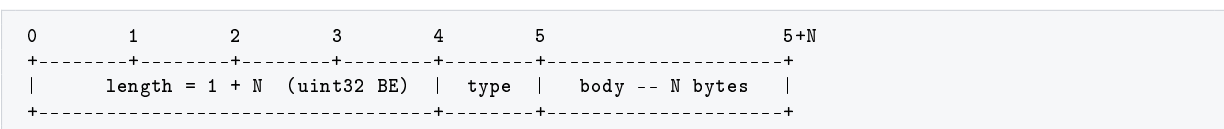
The protocol is split into two layers, so that neither has to change when the other does:

Layer	Responsibility
L2	Application grammar (LOGIN, MSG, WHO, QUIT)
L1	Record framing over the TCP byte stream

L1 knows nothing about chat; L2 knows nothing about how bytes are delimited, so later phases can insert processing between the two rather than modify either. The specification is in `docs/protocol.md`.

3.2.1 Message framing — deciding when a message is complete

TCP provides a reliable, ordered *byte stream*, not a message stream. A single `send()` may arrive split across several `recv()` calls, or concatenated with its neighbours. The protocol must therefore define its own message boundaries. Every record, in both directions and in every phase, is length-prefixed:



Reading a record is therefore a two-stage operation, each stage looping until satisfied: read exactly four bytes, decode the length with `ntohl()`, then read exactly that many further bytes. A message is complete when, and only when, the declared number of bytes has arrived.

Why not a newline delimiter. Delimiting messages with `\n` would be simpler, and entirely adequate for Phase 1 where every payload is printable text. It was rejected because the body will not stay printable: once payloads carry encrypted data they are uniformly distributed binary, so approximately one byte in 256 equals `0x0A` — a newline-framed reader would split messages at

random positions. The available escapes are to Base64-encode every record (a 33 % overhead, and framing is still needed on top) or to introduce an escaping scheme (whose bugs would be protocol-breaking). Length prefixing is binary-safe, so the framing written now does not have to be revisited later.

This was verified empirically rather than assumed: a test transmitted a message containing all 256 possible byte values, including 0x00, 0x0A and 0xFF, and confirmed byte-exact delivery.

Denial-of-service guard. A declared length above 16384 is rejected and the connection closed *before any allocation*. A length field trusted blindly is a one-packet denial of service: a peer claiming 0xFFFFFFFF would otherwise trigger a 4 GB allocation.

Record type byte. The single type byte distinguishes application data (0x02, the only type Phase 1 emits) from handshake records (0x01) and plaintext alerts (0x03). Reserving the codes now means that adding record kinds later requires neither a version bump nor a change to existing parsers.

3.2.2 Message grammar

The body of an application record is a single UTF-8 line: a verb followed by space-separated arguments.

Client → Server	Server → Client
LOGIN <username>	OK <username>
MSG <to> <text>	ERR <reason>
WHO	FROM <sender> <text>
QUIT	USERS <name> <name> ...
	INFO <text>

LOGIN registers a username so the server can route by name. It carries no password: the assignment never requires user authentication, and its reference to protecting a login exchange is explicitly conditional. Being ordinary application data, this message will be covered automatically by any protection later applied to the record body.

3.2.3 Routing, and more than two clients

The server maintains a username → socket map. On receiving MSG <to> <text> it looks up the recipient, forwards FROM <sender> <text> to that socket, and replies ERR no such user: <to> if the recipient is absent. Because routing is a map lookup rather than a hard-wired pairing, the design already supports an arbitrary number of simultaneous clients; the two required by Phase 1 are simply the case that is demonstrated. Broadcast is used only for join and leave notices.

The opacity rule. The server parses exactly two tokens of a MSG: the verb and the recipient. The message text is copied byte-for-byte and never inspected, validated or modified. This is a deliberate constraint rather than an implementation detail: keeping the relay ignorant of message content means the routing logic never has to change when the content does.

3.2.4 Command interface

The commands required from Phase 1 are implemented in the client and map onto the wire protocol as follows. They are user-interface concepts, deliberately kept distinct from the protocol messages.

User types	Client action	Sent on the wire
@bob hello	select bob, send	MSG bob hello
@bob	select bob	—
/chat bob	select bob	— (local state only)
/who	request the user list	WHO
/quit	disconnect and exit	QUIT
anything else	send to the selected partner	MSG <partner> <text>

The final row follows the specification literally: input matching no recognised command tag is treated as a plain chat message to the currently selected user. A mistyped `/quti` is therefore transmitted as chat text rather than rejected.

3.3 Implementation

3.3.1 Server

Single-threaded and `poll()`-driven, with all sockets non-blocking. Each connection owns an inbound record reader that buffers partial records, and an outbound byte queue flushed when the socket reports itself writable.

This structure is what prevents one client from stalling the others. A blocking design would have to wait inside `recv()` for the remainder of a half-delivered record, or inside `send()` for a client that has stopped reading; in either case every other client waits too. With buffering on both directions, a partial record simply remains buffered until more data arrives, and a slow reader only accumulates its own backlog — capped, so that a peer which never reads is dropped rather than consuming server memory without limit.

```
RecordReader::Status RecordReader::next(uint8_t& type, std::vector<uint8_t>& body)
{
    size_t avail = buf_.size() - off_;
    if (avail < HDR_LEN) return Status::NeedMore;

    uint32_t len;
    std::memcpy(&len, buf_.data() + off_, HDR_LEN);
    len = ntohl(len);

    if (len < 1 || len > MAX_RECORD) return Status::Malformed;
    if (avail < HDR_LEN + len) return Status::NeedMore;
    ...
}
```

Listing 1. Incremental record extraction (`proto.cpp`).

3.3.2 Client

The client must wait on two sources at once: the socket and the keyboard. Blocking on either must not make it deaf to the other, so it uses two threads — the main thread reads standard input and parses commands, while a reader thread blocks on the socket and prints what arrives. The socket itself remains blocking, since each thread blocks on its own source. Terminal output is serialised behind a mutex so that an arriving message cannot interleave inside a line being printed.

3.3.3 Robustness

Both programs ignore `SIGPIPE`, so that writing to a socket whose peer has departed returns an error instead of terminating the process. Beyond the framing guard already described, the server rejects application data sent before `LOGIN`, duplicate usernames, usernames outside `[A-Za-z0-9_]`, oversized payloads and unexpected record types, closing the connection in each case.

3.4 Verification

3.4.1 Test setup

The server ran on `vm1-server`, with clients `alice` on `vm2-client1` and `bob` on `vm3-client2`. Packet capture ran simultaneously on all three VMs (`server.pcap`, `c1.pcap`, `c2.pcap`), filtered to TCP port 5555 on `enp0s8`.

3.4.2 Functional result

Both clients connected simultaneously and exchanged messages in both directions. The session exercised `@username` message, `/who`, `/chat`, implicit send to the selected partner, join and leave notices, and `/quit` on both ends.

3.4.3 The server can read every message

The server logs every message it relays, with the full text. The complete log for the demonstration session (`evidence/phase1/server.log`):

```
[2026-09-04T00:54:46Z] START phase1 plaintext relay on 0.0.0.0:5555 (log: server.log)
[2026-09-04T00:54:52Z] CONN 10.10.0.11:58172
[2026-09-04T00:54:52Z] LOGIN alice from 10.10.0.11:58172
[2026-09-04T00:54:52Z] CONN 10.10.0.12:42680
[2026-09-04T00:54:52Z] LOGIN bob from 10.10.0.12:42680
[2026-09-04T00:54:54Z] WHO  alice -> [alice bob]
[2026-09-04T00:54:56Z] RELAY alice -> bob : "hey bob, are you there?"
[2026-09-04T00:54:58Z] RELAY bob -> alice : "yes alice, receiving you loud and clear"
[2026-09-04T00:55:00Z] RELAY alice -> bob : "this line goes to the selected partner"
[2026-09-04T00:55:02Z] RELAY bob -> alice : "the server can read every word of this"
[2026-09-04T00:55:05Z] RELAY alice -> bob : "plaintext is fully readable on the wire"
[2026-09-04T00:55:08Z] WHO  bob -> [alice bob]
[2026-09-04T00:55:09Z] QUIT  alice
[2026-09-04T00:55:09Z] CLOSE alice (quit)
[2026-09-04T00:55:10Z] QUIT  bob
[2026-09-04T00:55:10Z] CLOSE bob (quit)
```

Every RELAY line carries the complete message text, printed by the server at the moment it forwards the message. **The server is a full plaintext observer of the conversation: it can read the entire content of every message passing through it.**

This log line is the anchor for the assignment as a whole. The same statement, unchanged, will serve in a later phase as evidence that the server can no longer read anything. Identical code, opposite conclusion.

3.4.4 Wireshark — plaintext on the wire

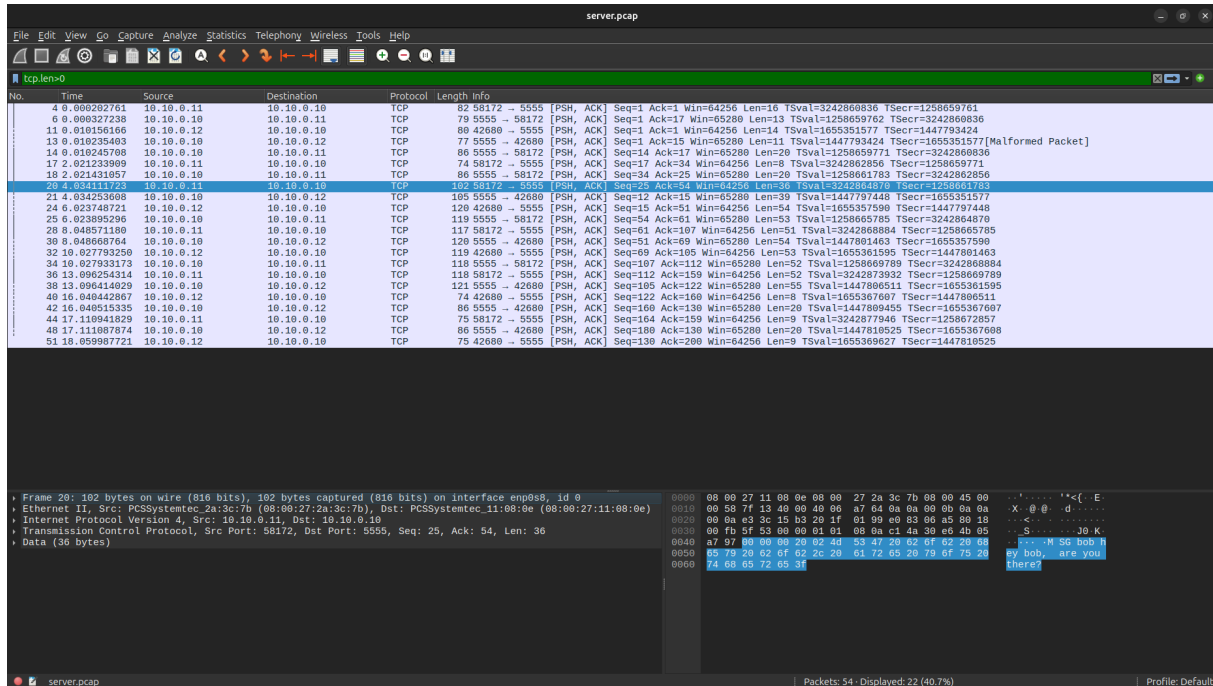


Figure 2. server.pcap opened in Wireshark, filtered to `tcp.len>0`. Frame 20 is selected: the bytes pane shows the payload `MSG bob hey bob, are you there?` directly readable in ASCII. The preceding five bytes are the record header — the 4-byte length prefix `00 00 00 20` (`32 = 1` type byte + `31` payload bytes) followed by the type byte `02`.

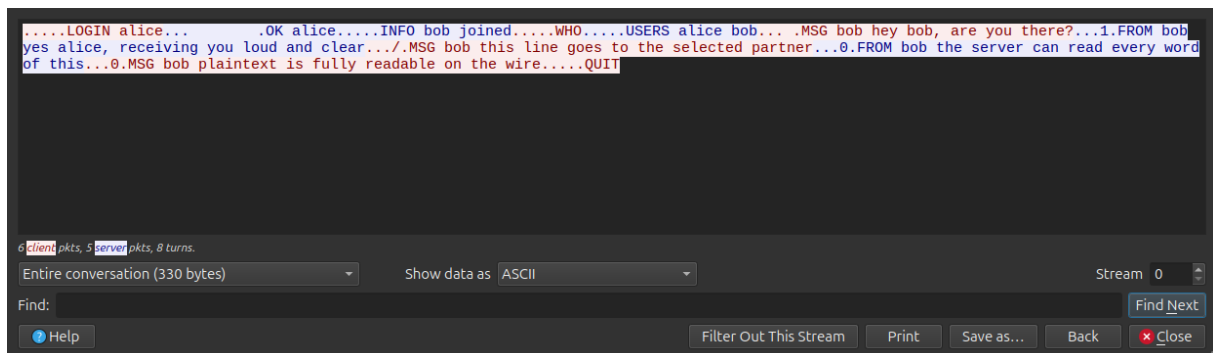


Figure 3. Follow → TCP Stream on server.pcap, stream 0 (client C1 ↔ server). The entire conversation is legible with no key and no decryption step: the login exchange, the user list, and every chat message in both directions. Client-to-server data is shown in red, server-to-client in blue.

The stream contents reproduce as:

```
.....LOGIN alice...      .OK alice.....INFO bob joined.....WHO.....USERS alice bob...
.MSG bob hey bob, are you there?...1.FROM bob yes alice, receiving you loud and clear
.../.MSG bob this line goes to the selected partner...0.FROM bob the server can read
every word of this...0.MSG bob plaintext is fully readable on the wire.....QUIT
```

The non-printable characters rendered as dots between messages are the framing itself — the four-byte length prefix and the one-byte record type — so the capture confirms both that the content is exposed and that the framing behaves as specified.

Anyone with access to the network segment reads the entire conversation, including the usernames. This is the baseline that the next phase must destroy: repeating this identical capture after adding confidentiality must show unreadable data at the same offsets.

3.5 Evidence index

All files are in `evidence/phase1/`.

File	Contents
<code>server.pcap</code>	Capture on <code>vm1-server</code> (54 packets, both client connections)
<code>c1.pcap</code>	Capture on <code>vm2-client1</code> (27 packets)
<code>c2.pcap</code>	Capture on <code>vm3-client2</code> (27 packets)
<code>server.log</code>	Server relay log with full message content
<code>wireshark-packet-list.png</code>	Wireshark packet view, plaintext in the bytes pane
<code>wireshark-follow-tcp-stream.png</code>	Follow → TCP Stream, whole conversation

3.6 Deliverables checklist

Requirement	Status
One server and two clients connected simultaneously over TCP	Done
Messages delivered <code>C1 → C2</code> and <code>C2 → C1</code> via the server	Done
Server relays each message as-is, without encryption	Done
Command interface <code>@user</code> , <code>/chat</code> , <code>/who</code> , <code>/quit</code>	Done
Server logs every relayed message; can read full content	Done
Wireshark capture with Follow → TCP Stream showing plaintext	Done
Protocol description: framing, message completeness, routing	Done

4 Building and Running

```
cd phase1 && make          # builds ./server and ./client

# on vm1-server (10.10.0.10)
./server --log server.log

# on vm2-client1 / vm3-client2
./client 10.10.0.10 alice
./client 10.10.0.10 bob

# packet capture, on any VM
sudo tshark -i enp0s8 -f 'tcp port 5555' -w /tmp/cap.pcap
```

Two environment notes recorded while capturing: `dumpcap` drops privileges even when invoked through `sudo`, so `tshark -w` cannot write into a home directory and must target a world-writable path; and Ubuntu’s AppArmor profile prevents `tshark -r` from reading capture files stored under `/home`.

5 Next Phase

Phase 2 adds confidentiality between each client and the server. The record framing, application grammar and routing logic described here are expected to carry over unchanged, with the work confined to the record body. The captures above will be repeated on the new protocol so that the two can be compared directly at the same offsets.

A Generative AI Usage

As required by assignment section 1.5, every prompt used during this assignment is recorded in

`docs/ai-prompts.md`

in the submitted repository. The file is maintained incrementally alongside the work rather than reconstructed afterwards: entries are added in the commit that contains the work they relate to, so the git history shows the prompts accumulating phase by phase. It is organised by phase, and each entry records the prompt together with what came out of it.

At the time of writing it covers Phase 0 (lab infrastructure, SSH access, network configuration, tooling and documentation) and Phase 1 (protocol design, implementation, evidence collection and this report). It will continue to grow as the remaining phases are implemented.